Blatt 07: Bike-Shop & Illusion Riddle (Refactoring, Javadoc)

Unterlagen

- PDF-Version

A07.1: Javadoc-Kommentare

Gute Javadoc-Kommentare schreiben erfordert Übung. Schauen Sie sich die in Commit Dungeon-CampusMinden/Dungeon/commit/46530b6 neu hinzugefügte Datei code/core/src/controller/ControllerLayer.java an.

Diskutieren Sie jeweils, was Ihnen an der Dokumentation dieser Klasse auffällt: Was gefällt Ihnen, was stört Sie? Schlagen Sie Verbesserungen vor.

DevDungeon: Illusion Riddle

Klonen Sie das Projekt DevDungeon und laden Sie es in Ihrer IDE als Gradle-Projekt. Betrachten Sie das Sub-Projekt "devDungeon". Dies ist ein von einem Studierenden (@Flamtky) erstelltes Spiel mit mehreren Leveln, in denen Sie spielerisch verschiedene Aufgaben *in-game* und *ex-game* lösen müssen.

A07.2: DevDungeon: Lösen des Illusion Riddle

Starten Sie den DevDungeon mit `./gradlew devDungeon:runDevDungeon`. Spielen Sie sich für diese Aufgabe durch das **dritte Level** ("Illusion Riddle")[1].

In diesem Level gibt es mehrere Räume, die durch Teleporter (statt Türen) miteinander verbunden sind. Beim Betreten dieser nur schwer erkennbaren dunkleren Bodenkacheln werden Sie in den nächsten Raum transportiert. Während dieser Mechanismus deterministisch ist, gibt es zusätzlich eine neue Monster-Art, die Sie mit fischähnlich aussehenden Geschossen angreift und bei einem Treffer in einen zufälligen Raum transportiert.

Im Level sind insgesamt drei Schalter verborgen, von denen Sie mindestens zwei finden und betätigen müssen, damit die weiteren Übergänge freigeschaltet werden. Die Reihenfolge spielt dabei keine Rolle.

Leider ist durch den starken *Fog of War* kaum etwas zu sehen. Auch die vielen Fackeln verbessern die Sicht nicht wirklich. Können Sie mit diesen Fackeln irgendwie interagieren?

Ziel ist es, den Weg zum letzten Raum des Levels zu finden und den dort wartenden Boss-Gegner zu besiegen.

Tipp: Es könnte hilfreich sein, sich eine Skizze der Räume anzufertigen. Diese sind in diesem Level bei jedem Start identisch.

Tipp: Es könnte in einem bestimmten Raum hilfreich sein, mehrfach im Kreis zu laufen …

[1] Das dritte richtige Level, also das dritte Level *nach* dem Demo-Level. Oder eben das vierte Level, wenn man das Demo-Level mitzählt :-)

Tipp: Eine Code-Analyse könnte helfen. Vielleicht können Sie durch Anpassungen im Code die Sicht verbessern oder die Gesundheit Ihres Helden verbessern oder die Teleportationsgeschosse unschädlich machen? Streng genommen ist das natürlich *cheaten*, aber da Sie ja Code lesen und anpassen üben, können wir im Rahmen dieser Lehrveranstaltung darüber hinwegsehen. Erklären Sie im Praktikum, welche Änderungen Sie wo und warum vorgenommen haben und was das bewirkt.

Hinweis: Aktuell ist das Projekt DevDungeon an einigen Stellen noch *Work-in-Progress*, beispielsweise fehlt häufig noch die Javadoc. Alle Gradle-Tasks, die von Checkstyle-Tasks abhängen (`checkstyleMain`, `check`, `build`, …) werden deshalb fehlschlagen. Sie können den DevDungeon aber wie oben beschrieben mit `./gradlew devDungeon:runDevDungeon` (bzw. über den Task `devDungeon:runDevDungeon` aus der IDE heraus) starten.

WICHTIG: Achten Sie bitte darauf, dass im Projektpfad **keine Leerzeichen** und keine Sonderzeichen (Umlaute o.ä.) vorkommen! Dies kann zu seltsamen Fehler führen. Bitte auch darauf achten, dass Sie als JDK ein **Java SE 21 (LTS)** verwenden.

A07.3: DevDungeon: Refactoring der Klasse IllusionRiddleLevel

Analysieren Sie die Klasse `IllusionRiddleLevel` im Package `level.devlevel`, insbesondere die beiden Methoden `IllusionRiddleLevel#onTick` und `IllusionRiddleLevel#lightTorch`:

1. Welche *Bad Smells* können Sie hier identifizieren?
2. Beheben Sie die Smells durch die **schrittweise Anwendung** von den aus der Vorlesung bekannten Refactoring-Methoden. Ergänzend zu der Übersicht aus der Vorlesung finden sie unter Refactoring Guru eine erweiterte Darstellung gängiger Refactoring-Techniken.

   Machen Sie pro Refactoring-Schritt einen Commit, und halten Sie alle Commits in einem Pull-Request fest. An diesem können Sie im Praktikum Ihr Vorgehen vorstellen.

Tipp: Schauen Sie schlechter Namensgebung, nach redundantem Code, nach übermäßig komplexer Logik, nach Code/Logik in der falschen Klasse (am falschen Ort), nach übermäßig vielen Parametern, nach fehlendem Javadoc, ….

Hinweis: Normalerweise erstellen Sie eine Testsuite, bevor Sie mit dem Refactoring beginnen. Leider ist durch die Abhängigkeit zu libGDX und der Game-Loop das Testen im (Dev-) Dungeon nicht trivial, so dass Sie hier ausnahmsweise direkt mit dem Refactoring loslegen dürfen und auf das Erstellen einer Testsuite verzichten können.

A07.4: Protector-Skill für Ihren Hero

Erstellen Sie einen neuen Protector-Skill für den Hero und weisen Sie diesen einer Taste zu. Bei Nutzung des Skills soll ein neues Protector-Monster an einer zufälligen Position in einem bestimmten Radius um den Helden erzeugt werden. Das Protector-Monster sucht das räumlich nächste Monster, nähert sich diesem automatisch bis auf Angriffsdistanz und greift dieses dann so lange mit Feuerbällen an, bis eines der beiden Monster keine Lebenspunkte mehr hat. Der Skill soll einen Cool-Down haben, d.h. er soll erst nach einer gewissen Zeit erneut benutzbar sein.

Nutzen Sie diesen neuen Skill im **dritten Level** ("Illusion Riddle") und schauen Sie, ob Sie dadurch leichter zum Ausgang des Levels kommen.

Hinweis: Erinnern Sie sich an die ECS-Architektur. Schauen Sie sich u.a. die Factory `HeroFactory` und den `FireballSkill` an.

A07.5: Refactoring im Bike-Shop

Forken Sie das Refactoring-Repo und erzeugen Sie sich eine lokale Arbeitskopie von Ihrem Fork. Analysieren Sie die Klassen im Package `refactoring`. Sie finden unübersichtlichen und schlecht strukturierten und schlecht benannten und schlecht dokumentierten Code.

1. Welche *Bad Smells* können Sie hier identifizieren?
2. Erstellen Sie eine Testsuite, um potentielle Verhaltensänderungen beim Refactoring identifizieren zu können.
3. Beheben Sie die Smells durch die **schrittweise Anwendung** von den aus der Vorlesung bekannten Refactoring-Methoden. Wenden Sie dabei *mindestens* die unten genannten Methoden an:

   - Extract Method/Class
   - Move Method/Field
   - Encapsulate Method/Field
   - Pull Up oder Push Down

   Ergänzend zu der Übersicht aus der Vorlesung finden sie unter Refactoring Guru eine erweiterte Darstellung gängiger Refactoring-Techniken.

   Machen Sie pro Refactoring-Schritt einen Commit, und halten Sie alle Commits in einem gemeinsamen Pull-Request fest. An diesem können Sie im Praktikum Ihr Vorgehen vorstellen.

Nach dem Refactoring sollte ein `./gradlew check` keine Probleme bzgl. Formatierung und Dokumentation[2] mehr finden.

Tipp: Schauen Sie schlechter Namensgebung, nach redundantem Code, nach übermäßig komplexer Logik, nach Code/Logik in der falschen Klasse (am falschen Ort), nach übermäßig vielen Parametern, nach fehlendem Javadoc, ...

[2] d.h. Sie müssen gegebenenfalls auch Javadoc ergänzen ...

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** e48b588 (Homework: improve wording (B07), 2025-04-18)